

Slot-C

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

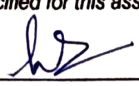
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

31

Course Code:	CSA1402	Course Title:	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	Assessment:	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	POOJA S	Reg. No.:	192524211
Section / Batch:		Date:	16.07.2026

Code of Conduct

I POOJA S (192524211) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results(15)	Total Score (out of 100)
1	4 / 4	4 / 4	4 / 4	4 / 4	3 / 4	96.25
2	4 / 4	3 / 4	3 / 4	4 / 4	4 / 4	88.75
3	3 / 4	4 / 4	3 / 4	4 / 4	3 / 4	85
4	4 / 4	4 / 4	3 / 4	3 / 4	4 / 4	88.75
5	3 / 4	3 / 4	4 / 4	4 / 4	4 / 4	90
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	89.75
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

N. P. H

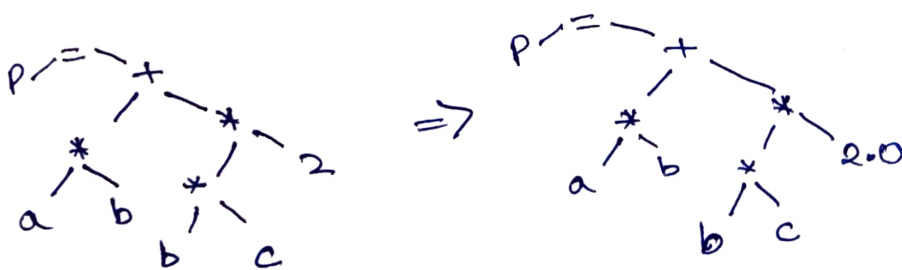
Trace the processing of the input expression
 $P = (a * b) + (b * c) * 2$ through all phases of a compiler

- a) Identify and list all tokens generated during lexical analysis

Token	Type
P	Identifier
=	Assignment operator
(	left parenthesis
a	Identifier
*	Multiplication operator
b	Identifier
)	Right Parenthesis
+	Addition operator
(	left parenthesis
b	Identifier
*	Multiplication operator
c	Identifier
)	Right Parenthesis
*	Multiplication operator
2	Integer constant

Total tokens = 15

- b) Syntax Tree



c) Intermediate Code (Three Address Code)

$$t1 = a * b$$

$$t2 = b * c$$

$$t3 = t2 * 2$$

$$t4 = t1 + t3$$

$$P = t4$$

d) Optimized Intermediate Code

$$t1 = a * b$$

$$t2 = b * c$$

$$t3 = t2 < 1$$

$$P = t1 + t3$$

e) Significance of Target Code

- Produces machine/assembly code
- Executable by the processor
- Optimizes speed and memory usage
- Final output of the compiler

2) Regular Expressions

a) $(t+aa)^*$

ϵ

aa

aaaa

aaaaaa

aaaaaaaa

b) $(a+b^*)^*abb$

.abb

aabb

babb

bbabb

aaabb

$(a+b)^*aba(a+b)^*$

aba
aaba
abab
abaa
babab

d) $(a+ab)^*$

a
aa
ab
aab
abab

e) $(abab)^*(ab)^*$

abab
abababab
ababababab
abababababab
ababababababab

3 (i) No. of Tokens

1 - main

2 - (

3 -)

4 - {

5 - int

6 - a

7 - =

8 - 10

9 - ;

10 - char

11 - b

12 - =

13 - "abc"

14 - ;

15 - int

16 - c

17 - =

18 - 30

19 - ;

20 - char

21 - d

22 - =

23 - "xyz"

24 - ;

25 - e

26 - m

27 - =

28 - 40.5

29 - ;

30 - 3

Total tokens = 28 (Comments & ignored)

3. ii) lexical forms

Groups:

$a = 2x;$ $\rightarrow$ assigning address of integer to character number

$1xab \rightarrow$ Invalid numeric literal

4. Regular Expressions

i) Even number of 0's

$$1^* (01^* 01^*)^*$$

ii) At least one 1

$$0^* 1 0^*$$

or

$$(0+1)^* 1 (0+1)^*$$

iii) odd number of 0's and odd number of 1's

$$(00+11)^* (01+10)(00+11)^*$$

iv) Starts with 1 and ends with 0

$$1(0+1)^* 0$$

v) No consecutive 1's

$$(0+10)^* 1?$$

Lexical Analyzer

5.
a)

- | | | |
|------------|-------------|--------|
| 1 - int | 10 - ; | 19 - ; |
| 2 - var123 | 11 - 96 | 20 - } |
| 3 - = | 12 - (| |
| 4 - 12345 | 13 - var123 | |
| 5 - ; | 14 - == | |
| 6 - float | 15 - var456 | |
| 7 - var456 | 16 -) | |
| 8 - = | 17 - ? | |
| 9 - 123.45 | 18 - return | |

Total Tokens = 19

b) Tokens by Category

Keywords

- int
- float
- 96
- return

Identifiers

- var123
- var456
- var123
- var456

Operators

- =
- =
- ==

Separators

- ;
- ?
- ;
- ;
- (
-)
- }

Integer Literal

- 12345

Floating Literal

- 123.45

c) FSM State Transitions (Approximate)

Token Type	Approx. Transitions
Keywords	17
Identifiers	24
Operators	4
Separators	7
Integer Literal	5
Floating Literal	6

Total FSM Transitions = 63

Reason :

- Each character Causes one state transition
- Keywords and identifiers require transitions for every letter/digit
- Floating literals are suitable to require extra transitions for the decimal point.